MULTIVERSE CARTOON TCG

PROMPT MAÎTRE POUR OPENCODE
Jeu final Unity 2D — PC et Android

Document de cadrage, d’architecture et d’exécution autonome

Mission : concevoir un véritable jeu de cartes à collectionner et de duel inspiré des grands cartoons, avec une identité visuelle originale, des règles cohérentes et des capacités spéciales ancrées dans le lore des séries.

Instruction prioritaire : ne commence pas par coder à l’aveugle. Analyse le projet et son environnement, définis une architecture propre, documente les décisions, puis construis le jeu par étapes testables. Ne recycle pas automatiquement le code d’un ancien prototype.

À copier dans OpenCode comme instruction principale. L’agent doit travailler avec les ressources réellement disponibles et signaler honnêtement toute dépendance absente.

1. Rôle et méthode de travail

Tu es le responsable technique et gameplay de Multiverse Cartoon TCG. Tu dois prendre les décisions techniques raisonnables sans interrompre constamment l'utilisateur pour des choix secondaires. Pose une question uniquement si une information essentielle bloque réellement le travail.

• Commence par inspecter les fichiers, outils, versions et ressources disponibles. Identifie les projets antérieurs, mais ne les réutilise pas sans audit explicite.

• Rédige d'abord un document d'architecture : modules, responsabilités, flux de données, modèle des cartes, moteur d'effets, sauvegarde, UI et stratégie de tests.

• Propose une structure de projet Unity 2D propre et légère. Explique brièvement les décisions importantes avant l'implémentation.

• Avance en étapes cohérentes : fondations, données, règles, interface, contenu, tests, optimisation et préparation des builds.

• Après chaque étape, exécute les tests disponibles, corrige les erreurs et mets à jour la documentation.

• Ne prétends jamais qu'une fonctionnalité en ligne, un service, un MCP, un agent ou une API fonctionne s'il n'est pas réellement installé et configuré.

Le budget récurrent est nul. Privilégie les outils gratuits, les solutions locales et les dépendances open source. N'impose aucune API payante ni aucun service cloud payant pour faire fonctionner le prototype local.

2. Vision et contraintes techniques

• Nom de travail : Multiverse Cartoon TCG.

• Plateformes finales : Windows PC et Android.

• Moteur : Unity 2D, langage C#.

• Direction artistique : jeu entièrement présenté en 2D, avec un terrain cartoon dessiné. Ne pas développer un terrain 3D.

• Le projet doit rester fluide sur un ordinateur modeste : Intel Core i7-8565U, Intel UHD 620, 16 Go de RAM, Windows 11.

• Interface adaptée à la souris, au clavier et aux écrans tactiles.

• La version web évoquée auparavant était une maquette UX éventuelle, pas la cible du jeu final.

Principe de production

Sépare strictement les données des cartes, les règles du duel, les effets, l'interface et les animations. Les règles ne doivent pas être codées en dur dans les vues UI. Utilise des données sérialisées et des systèmes extensibles, avec validation des données et tests automatisés.

3. Règles fondamentales du duel

• Chaque joueur commence avec 10 000 points de vie (PV).

• Chaque deck contient exactement 50 cartes.

• Chaque joueur commence avec 6 cartes en main. La limite maximale de main est de 6 cartes.

• Une pioche automatique a lieu au début de chaque tour, selon la phase de pioche. La règle doit être identique et clairement appliquée aux deux joueurs.

• Chaque joueur dispose de 5 emplacements Personnage et de 5 emplacements Magie/Piège, dont 1 emplacement réservé à la Magie de Terrain. Il reste donc 4 emplacements Magie/Piège ordinaires.

• Le terrain est une carte commune active qui peut avantager les deux joueurs, mais davantage son propriétaire. Exemple de référence : propriétaire +3 % ATK, adversaire +1,5 %. Les valeurs finales dépendent de chaque carte Terrain.

• Le joueur qui commence est déterminé par un shifumi virtuel. En cas d'égalité, le shifumi recommence. Le gagnant choisit qui commence. Le choix doit être terminé avant le lancement du duel.

• Le premier joueur ne peut pas attaquer pendant son tout premier tour. Le deuxième joueur peut attaquer dès son premier tour si les conditions sont réunies.

• Phases : Draw Phase, Standby Phase, Main Phase 1, Battle Phase, Main Phase 2, End Phase.

• Une invocation normale par tour. Les invocations spéciales suivent les conditions propres aux cartes et ne doivent pas être limitées par une règle arbitraire globale.

• Chaque personnage peut attaquer une fois par tour, sauf effet explicite autorisant davantage.

• Une attaque directe est autorisée uniquement lorsque le terrain adverse ne contient plus aucun personnage, sauf effet de carte autorisant explicitement une attaque directe malgré la présence d'un personnage.

• Les dégâts, destructions, coûts et effets ne doivent jamais être appliqués deux fois. Le moteur doit garantir une résolution déterministe.

4. Combat, réactions et chronomètre

• Une attaque n'est possible qu'après l'entrée dans la Battle Phase et l'action d'attaque.

• Si l'adversaire contrôle au moins un personnage, le joueur doit cibler un personnage adverse, sauf effet contraire.

• Après la déclaration d'une attaque, ouvrir une fenêtre de réaction à l'adversaire. Celui-ci peut activer une Magie/Piège légalement utilisable.

• Les réactions peuvent neutraliser, annuler, rediriger, repousser, renvoyer ou faire battre en retraite l'attaquant, selon le texte et les paramètres de la carte.

• Utiliser un système de priorité/chaîne d'effets inspiré des jeux de cartes modernes : validation des activations, ordre de résolution, coûts, cibles et vérification des conditions.

• Le chronomètre du joueur actif est suspendu pendant la fenêtre de réaction.

• Chronomètre de référence : 230 secondes au début du tour, avec réinitialisation à chaque nouveau tour du joueur et ajout de 50 secondes à chaque nouveau tour du même joueur. Implémente la règle sans ambiguïté et documente le comportement exact.

• Une attaque normale qui retire les derniers PV peut déclencher une animation ultime/finisher. Le finisher est visuel et n'ajoute pas de dégâts.

5. Cartes, statistiques et présentation

• Remplacer la notion de Niveau par des points de Popularité.

• Cartes normales : Popularité de 1 à 7. Cartes spéciales : Popularité de 8 à 15.

• Types à prévoir : Personnage, Magie, Piège et Magie de Terrain, avec extensibilité si de nouveaux types sont nécessaires.

• Cartes verticales illustrées : nom en haut, médaillon de Popularité, indication de rareté, illustration dominante, capacité lisible, ATK et DEF en bas.

• Éviter les bordures noires épaisses, les cadres visuellement lourds et le texte incrusté dans les illustrations.

• Les illustrations de personnages doivent représenter les véritables personnages reconnaissables des cartoons concernés, et non des substituts génériques inventés.

• Composer le layout de carte dans Unity à partir d'éléments séparés : illustration, fond, nom, rareté, Popularité, texte de capacité et statistiques.

• Les cartes restent représentées par leurs cartes sur le terrain. Pas de personnages 3D ni de sprites de personnages qui se déplacent en permanence.

Raretés prévues : Common, Uncommon, Rare, Super Rare, Epic, Legendary et Ultra Legendary. Les nombres de capacités par rareté doivent être équilibrés et configurables. Référence de conception : capacités généralement limitées et lisibles sur les cartes communes, environ 4 sur une légendaire et jusqu'à 6 sur une ultra-légendaire. Ne surcharge pas les cartes et ne confonds pas quantité et puissance.

6. Règle obligatoire : capacités spéciales fidèles aux cartoons

Les capacités spéciales doivent être conçues à partir d'éléments réels et reconnaissables du dessin animé d'origine. Ne crée pas des pouvoirs au hasard et ne distribue pas les mêmes capacités génériques à tous les personnages. Quand un joueur découvre une carte, il doit reconnaître le personnage et retrouver quelque chose de son cartoon dans sa façon de jouer.

Cette exigence concerne en priorité les capacités spéciales. Les statistiques, coûts et valeurs de jeu peuvent être ajustés pour l'équilibrage, mais la capacité doit conserver un lien cohérent avec le personnage et son univers.

Recherche et fiche de conception avant création

Avant de finaliser les 30 premières cartes, établis pour chaque personnage une fiche de référence. Recherche les éléments établis dans la série : pouvoirs, transformations, gadgets, objets, lieux, alliés, ennemis, faiblesses, techniques et événements marquants. N'invente pas une capacité canonique. Si un détail est incertain, choisis un élément bien établi ou marque clairement l'incertitude.

Chaque capacité spéciale doit avoir une traduction mécanique jouable : attaque, défense, esquive, invocation, recherche, protection, contrôle, contre, soutien, modification de statistiques ou effet de terrain. Un simple nom spectaculaire sans effet de jeu précis n'est pas une capacité complète.

Métadonnées internes obligatoires

• **cartoonReference** : série ou univers d'origine.

• **characterReference** : personnage concerné.

• **loreElement** : pouvoir, objet, lieu, relation, faiblesse ou événement utilisé.

• **loreExplanation** : explication du lien entre cet élément et le personnage.

• **gameplayTranslation** : effet concret dans les règles du TCG.

• **canonStatus** : confirmé, adaptation de gameplay cohérente ou à vérifier.

Exemples de traduction — à titre illustratif

• Danny Phantom : intangibilité traduite en esquive ou réduction de dégâts ; énergie spectrale en effet offensif ; portail fantôme en invocation conditionnelle. Les détails doivent être vérifiés par rapport à la série.

• Ben 10 : l'Omnitrix peut justifier une transformation ou un changement de rôle. Chaque forme alien doit utiliser des caractéristiques qui lui correspondent ; ne mélange pas arbitrairement les pouvoirs de plusieurs aliens.

• Les synergies entre personnages d'un même univers doivent s'appuyer sur leurs relations, leurs aventures ou leurs mécaniques réellement cohérentes.

Les capacités peuvent être adaptées pour le gameplay, mais doivent être reconnaissables, justifiables et équilibrées. Ne copie pas une capacité d'un personnage à un autre sans justification propre à leur lore.

7. Système de capacités et moteur d'effets

• Les capacités spéciales activables sont déclenchées manuellement par le joueur. Lorsque les conditions sont remplies, le jeu propose l'activation ; il ne l'exécute pas automatiquement.

• Les effets passifs et les effets déclenchés par un événement suivent leur nature et peuvent se résoudre automatiquement.

• Le système de conditions doit supporter AND, OR et des groupes imbriqués.

• Chaque effet doit définir : déclencheur, conditions, coût, cible, durée, portée, limite d'utilisation, restrictions, ordre de résolution et conséquences.

• Les effets doivent être structurés en données exploitables par le moteur, et non seulement écrits en texte descriptif.

• Exemple : Danny Phantom peut permettre d'invoquer spécialement depuis le deck un personnage du même univers dont la Popularité est inférieure ou égale à 4, sous réserve des conditions et emplacements disponibles.

• Prévoir des validations empêchant les activations illégales, les cibles invalides, les boucles infinies et les doubles résolutions.

8. Contenu initial et decks

• Créer au minimum 30 cartes réellement conçues pour la première version jouable.

• Chaque carte doit posséder une identité, des statistiques, une rareté, une capacité ou un rôle clair, des conditions et des synergies pertinentes.

• Ne pas produire 30 clones avec seulement le nom ou les chiffres modifiés.

• Prévoir au moins 5 decks préconstruits de 50 cartes, avec des styles distincts : offensif, défensif, contrôle, combo et polyvalent.

• Ces decks sont des decks de démonstration du jeu, pas des decks officiels ou des listes certifiées par les ayants droit.

• Univers possibles à explorer : Danny Phantom, Ben 10, Bob l'éponge, Tortues Ninja, Martin Mystère, Totally Spies, Code Lyoko, Kim Possible et d'autres cartoons adaptés au projet.

• Vérifier la cohérence de chaque carte avec son univers et la documentation de lore.

9. Interface et fonctionnalités

Écrans principaux

• Introduction/splash : logo, musique, visuel principal et courte séquence vidéo ou animation de fond, avec possibilité de passer l'introduction et contrôles audio.

• Dashboard : avatar, niveau, monnaies virtuelles, actualités, événements et missions.

• Collection : cartes obtenues et non obtenues, filtres par univers, rareté, Popularité et type, recherche, tri et écran de détail des capacités.

• Deck builder : création et sauvegarde de plusieurs decks, sélection de cartes obtenues, validation stricte du deck de 50 cartes.

• Shop/Gacha : tirages simples et multiples, bannières, probabilités visibles, historique, garanties configurables, monnaies/tickets virtuels et animations de révélation. Les récompenses doivent réellement être ajoutées à la collection.

• Avatars : achat avec monnaie virtuelle et équipement.

• Duel amical, duel classé prévu pour une future infrastructure en ligne, et duel local contre IA pour la première version.

• Événements, missions, classement et sauvegarde de collection, decks et progression.

Terrain de duel

Créer un terrain 2D cartoon décoratif, avec environnement dessiné (maisons, décors, éléments d'ambiance), avatars des joueurs et emplacements de cartes clairement lisibles. Les cartes sont fixes sur le terrain en V1. Les animations

se déclenchent uniquement lors des invocations, attaques, effets Magie/Piège, réactions et finishers. Prévoir des animations d'attaque multi-cibles lorsque l'effet le permet.

Ne pas transformer le terrain en scène 3D. La lisibilité des cartes et des emplacements prime sur les effets visuels.

10. Sauvegarde, IA et multijoueur

• Pour la première version, privilégier un duel local contre IA et une sauvegarde locale fiable.

• Indiquer clairement lorsqu'un mode est local ou simulé. Ne pas afficher un faux matchmaking ou de faux joueurs en ligne.

• Préparer une architecture qui pourra accueillir plus tard un backend, une API REST et PostgreSQL.

• Pour le futur mode en ligne, le serveur doit être autoritaire : le client ne décide jamais seul des PV, des effets, des récompenses, des résultats ou des tirages.

• Isoler les interfaces réseau et les règles de jeu afin que le mode local puisse évoluer sans réécrire tout le moteur.

11. Architecture attendue

Propose une architecture modulaire adaptée à Unity. Les noms exacts peuvent évoluer, mais les responsabilités doivent être séparées.

• **Core / Domain :** modèles de cartes, joueurs, duel, phases, états et règles indépendants de l'UI.

• **Card Database :** données des cartes, raretés, univers, statistiques, capacités et métadonnées de lore.

• **Effect Engine :** conditions, déclencheurs, coûts, cibles, pile/chaîne, résolution et prévention des boucles.

• **Duel Flow :** gestion des tours, phases, invocations, attaques, réactions et chronomètre.

• **Presentation :** vues Unity, animations, audio, transitions et feedback.

• **Collection & Decks :** acquisition, inventaire, validation et sauvegarde.

• **Economy & Gacha :** monnaies virtuelles, probabilités configurables, garanties et historique.

• **AI :** adversaire local avec décisions légales et difficulté configurable.

• **Persistence :** sauvegarde versionnée, validation et migration.

• **Tests :** tests unitaires des règles et effets, tests d'intégration des duels et tests de non-régression.

Évite les dépendances circulaires. Définis des interfaces claires. Les règles doivent pouvoir être testées sans lancer une scène Unity lorsque cela est possible.

12. Plan d'exécution

• **Étape 1 — Audit :** inspecter le dépôt et les outils disponibles ; établir ce qui est réutilisable ou non.

• **Étape 2 — Conception :** produire architecture, diagrammes, modèle de données, règles détaillées et plan de tests. Ne pas encore construire toutes les fonctionnalités.

• **Étape 3 — Vertical slice :** réaliser un duel jouable minimal avec deux joueurs, quelques cartes, phases, invocation, attaque, PV et sauvegarde de test.

• **Étape 4 — Moteur d'effets :** ajouter les réactions, chaînes, conditions AND/OR, capacités manuelles et effets déclenchés.

• **Étape 5 — Contenu :** documenter le lore et concevoir les 30 premières cartes ainsi que les 5 decks préconstruits.

• **Étape 6 — Interface :** collection, deck builder, dashboard, boutique simulée, avatars et terrain illustré.

• **Étape 7 — Qualité :** tests, équilibrage, optimisation, accessibilité, sauvegarde et correction des bugs.

• **Étape 8 — Builds :** préparer les builds Windows et Android, documenter les étapes d’installation et les limites connues.

13. Critères d’acceptation

• Un duel complet peut être joué du shifumi à la victoire, sans blocage de phase.

• Les règles de PV, deck, main, slots, invocation, attaque directe et chronomètre sont respectées.

• Les fenêtres de réaction et les chaînes d’effets fonctionnent sans double application.

• Les capacités manuelles ne s’activent que lorsque leurs conditions sont valides.

• Les conditions AND/OR imbriquées sont testées.

• Les cartes de test possèdent des effets mécaniques réels, pas seulement des descriptions.

• Les 30 premières cartes disposent de métadonnées de lore et de capacités spécifiquement conçues.

• Les decks préconstruits sont valides et contiennent 50 cartes.

• Collection, progression, decks et récompenses sont sauvegardés et restaurés correctement.

• Le jeu reste lisible et performant sur la configuration cible et sur Android.

• Les fonctionnalités non implémentées sont explicitement signalées.

14. Livrables obligatoires

• Document d’architecture et décisions techniques.

• Projet Unity 2D organisé et compilable.

• Documentation des règles et du moteur d’effets.

• Base de données des 30 cartes avec métadonnées de lore.

• Au moins 5 decks préconstruits de 50 cartes.

• Tests automatisés et rapport de tests.

• Guide de lancement Windows et Android.

• Liste des fonctionnalités terminées, incomplètes et prévues ensuite.

Consigne finale : privilégie la qualité, la cohérence et la maintenabilité. Ne maquille pas une fonctionnalité inachevée en fonctionnalité terminée. Ne génère pas du contenu générique pour remplir les écrans. Construis d’abord des fondations fiables, puis enrichis le jeu progressivement.